

Guía formativa curso de Angular





Tabla de Contenido

1	Instalar el entorno de trabajo.....	4
1.1	Otras instalaciones recomendadas (Extensiones de VSCode).....	5
1.1	Compatibilidad entre versiones.	5
2	Arrancando nuestro primer proyecto de angular	6
2.1	Creación del proyecto.....	6
2.2	Archivos básicos en una aplicación en Angular	7
3	Estructura Base de la aplicación	8
4	Aplicación Angular: Pensando en componentes.....	9
4.1	Estructura base del componente en código.....	9
4.2	Creación de Componentes	9
4.3	Componentes Estandar vs Standalone	10
4.4	Estructurando la aplicación en directorios / módulos	11
5	Navegación en Angular. Entender el routing	12
5.1	¿Qué es el Router?.....	12
5.2	Elementos básicos en el router	12
5.3	¿Dónde y cómo se definen estas rutas?	12
5.4	Navegar desde le componente (RouterLink) y mostrar el link activo (RouterLinkActivate)	13
6	Ejercicio: Generar componentes y visualizarlo en nuestra aplicación.....	14
7	Dar contenido a nuestros componentes.....	15
7.1	Principales directivas en Angular	15
7.2	Decorador @ViewChild / viewChild() de Angular	16
7.3	Uso de estructuras de datos: interfaces, enum y arrays	17
7.4	Utilidades para la depuración del código.	19
7.5	Los Pipe en Angular.....	19
7.6	Ejercicio: Crear una pantalla con un listado de clientes	20
7.7	Ejercicio 2: Modificar pantalla con un listado de clientes	20
7.8	Cuadro resumen interacción con el html	20
7.9	Introducción al uso de señales	21
8	Que son los servicios	22
8.2	Creando un servicio	22
8.3	Trabajar con el servicio creado	22
8.4	Ejercicio: Servicio Básico.....	23
9	Uso de Servicios para el consumo de APIs por peticiones HTTP	24
9.1	Ejemplo práctico de cómo usar un servicio conectado con el consumo de una API	24
10	Comunicación entre componentes.....	26
10.1	Comunicación con componente hijo (input)	26
10.2	Comunicación con el componente padre (output)	28



10.3	Ejercicio: Pantalla de listado de clientes	31
10.4	Ejemplo 2	32
	Crear una nueva página de clientes con 2 componentes hijos	32
-	Lista-clientes (listado de clientes)	32
-	Ficha-cliente (ficha con los datos del cliente seleccionado)	32
	Objetivo: Comunicación entre ellas a través del componente padre	32
11	Que son los módulos en Angular	33
11.1	Esquema organizativo mediante modulos.	33
11.1	Creación de los Modulos	34
11.2	Comunicación entre Módulos	34
11.3	Como organizar el contenido de un modulo	34
11.4	Ejercicio práctico:	36
12	Navegación en Angular (II). Ampliación del uso de rutas.	37
12.1	LazyLoading y rutas hijas	37
12.2	Paso de parámetros en las rutas	38
13	Ciclo de vida de un componente Angular	39
14	Formularios Basados en Plantillas (ngForm)	42
14.1	Conceptos principales	42
14.2	Ejemplo básico	42
15	Formularios Reactivos en Angular	43
15.1	Conceptos principales	43
15.2	Ejemplo básico	43
15.3	Elementos para definir y crear un formulario.	44
15.4	Validación de datos del formulario	45
15.5	Mostrar Errores	45
15.6	Modificación de valores del formulario desde TypeScript	45
15.7	Escuchar Cambios en los valores del formulario	45
15.8	Envío del Formulario	46
16	Diferencia Formularios plantilla (NgForm) vs Formularios Reactivos	47
17	Alias: Utilidad para mejorar la legibilidad de nuestro código	49



SECCION 1: Empezando a trabajar con Angular

1 Instalar el entorno de trabajo.

Para empezar a trabajar con Angular necesitaremos prepara primero nuestro entorno de trabajo, para ello debemos instalar en nuestra maquina:

1. NVM (Node Version Manager)

Esta no es una instalación necesaria, pero si altamente recomendada ya que nos permite trabajar fácilmente con múltiples versiones de node.js. y con ello de Angular. (Permite usar la versión exacta que pide cada proyecto).

En caso de utilizar esta herramienta se recomienda no tener ninguna versión previa de node y en caso afirmativo desinstalarla previamente. Además, debes saltarte el paso 2 (instalar Node.js) tal y como se describe y realizar la instalación de las diferentes versiones de node.js a través de nvm para que permita controlar dichas versiones.

Para instalarlo descargamos el paquete instalador de la web (<https://www.nvmnode.com/es/>) y los ejecutamos en modo administrador. (instalación con siguiente)

Una vez instalado usaremos los siguientes comandos en la consola de comandos para su gestión:

- **nvm install** <versión_node>: Descarga una versión nueva.
- **nvm ls**: Muestra las versiones de node que ya tienes instaladas.
- **nvm use** <versión>: Activa una versión node indicada en la terminal.

2. Instalar Node.js

Angular utiliza Node.js para ejecutar las herramientas de desarrollo. Podemos descargar node.js directamente de la web (<https://nodejs.org/es>).

Descargamos el paquete (Se recomienda instalar una versión TLS) y realizamos una instalación en modo administrador básicamente dando a siguiente.

Comprobar la instalación (usamos los siguientes comandos en la consola)

- `node -version` o `node -v`
- `npm -versión` o `npm -v`

3. Instalar AngularCli

Angular CLI permite crear, ejecutar, compilar y gestionar proyectos Angular

La instalación la realizamos en consola de comandos ejecutada en modo administrador a través del comando npm

- **npm install -g @angular/cli**
- Para una versión concreta usamos **@angular/cli@Version**

Comprobar que funciona

- `ng versión` o `ng v`

4. Instalar un editor de código

En nuestro caso utilizaremos el editor gratuito de Microsoft “Visual Studio Code”, que se puede descargar gratuitamente de la web



5. Configurar Git

- `git config --global user.name "Nombre Apellido"`
- `git config --global user.email "tu correpo@correo@empresa.com"`

1.1 Otras instalaciones recomendadas (Extensiones de VSCode)

A la hora de trabajar con Angular se recomienda que instales algunas extensiones de VSCode que nos van a hacer la vida mucho más fácil a la hora de desarrollar nuestra aplicación.

Algunas recomendaciones son:

Extensiones Visual Studio Code

Extensión	Funcionalidad
Angular Language Service	Entender sintaxis de angular
Angular Snippets	Escribir componentes y crear servicios muy rápido
Angular Schematics	Crear componentes dentro de nuestras carpetas del proyecto
Angular 2 Inline	Ver la sintaxis acorde a lo que estamos escribiendo
Auto Close Tag	Cerrar etiquetas de forma automática
Auto import	Solventa problemas con las auto-importaciones
Auto Rename Tag	Renombramiento de etiquetas (cierre)
Error Lens	Ver los errores sin necesidad de dejar el cursor encima del error
Paste JSON as Code	Interfaces acordes a objetos json
Editor Config for VSCode	Tener el editor como se recomienda por parte de angular
Better Comments	Tener los comentarios clasificados y con colores
Tailwind CSS IntelliSense	Ayuda con la gestión de estilos de Tailwind CSS
Pretty TypeScript Errors	Mejorar la visualización de errores de typeScript
Material Icon Theme	Pone iconos identificativos en carpetas y archivos según funcion

1.1 Compatibilidad entre versiones.

Este es un punto muy importante ya que no todas las versiones de Node.js son compatibles con todas las versiones de Angular CLI. La compatibilidad se define principalmente por la versión de Angular que utiliza el proyecto, por ello, Primero determino la versión de Angular del proyecto → después elijo una versión de Node.js compatible.

Un error común es instalar una versión global del CLI diferente a la del proyecto, por ejemplo, `npm install -g @angular/cli`, me instalara la versión más reciente de angular, que puede no ser compatible con la versión de node instalada en mi equipo.

Recuerda que puedes indicar la versión a instalar de angular con `npm install -g @angular/cli@VersioAinstalar`

Aquí es donde toma importancia la gestión de paquetes de node a través de la herramienta nvm vista al principio de este documento, ya que me va a permitir tener diferentes versiones de node y seleccionar la versión que sea compatible con mi proyecto de angular. Ejemplo:

- Proyecto A → Angular 18 → Node 20
- Proyecto B → Angular 16 → Node 18
- Proyecto C → Angular 20 → Node 22



Puedes ver la compatibilidad de versiones en la página de angular (<https://angular.dev/reference/versions>)

2 Arrancando nuestro primer proyecto de angular

Una vez que ya tenemos nuestro entorno, podemos crear nuestro primer proyecto en angular

2.1 Creación del proyecto

1. Crear un proyecto en Angular

- Normalmente a través de la consola de comandos nos situamos en el directorio que va a contener nuestro proyecto y ejecutamos el comando Angular
 - `ng new mi-aplicacion`
- Entramos en el directorio creado <mi-aplicación> y hay estarán todos los archivos del proyecto
 - Arancamos VSCode escribiendo “code .” o bien abrimos la carpeta en VSCode
- **NOTA:** Para trabajar sobre un proyecto existente:
 - `git clone <repositorio>`
 - `cd mi-aplicacion`
 - `npm install` (para reconstruir el contenido de `node_modules`, que contiene las librerías y dependencias que necesita mi proyecto)

2. Ejecutar la aplicación

Si queremos ejecutar la aplicación tenemos que utilizar el comando angular `ng serve`

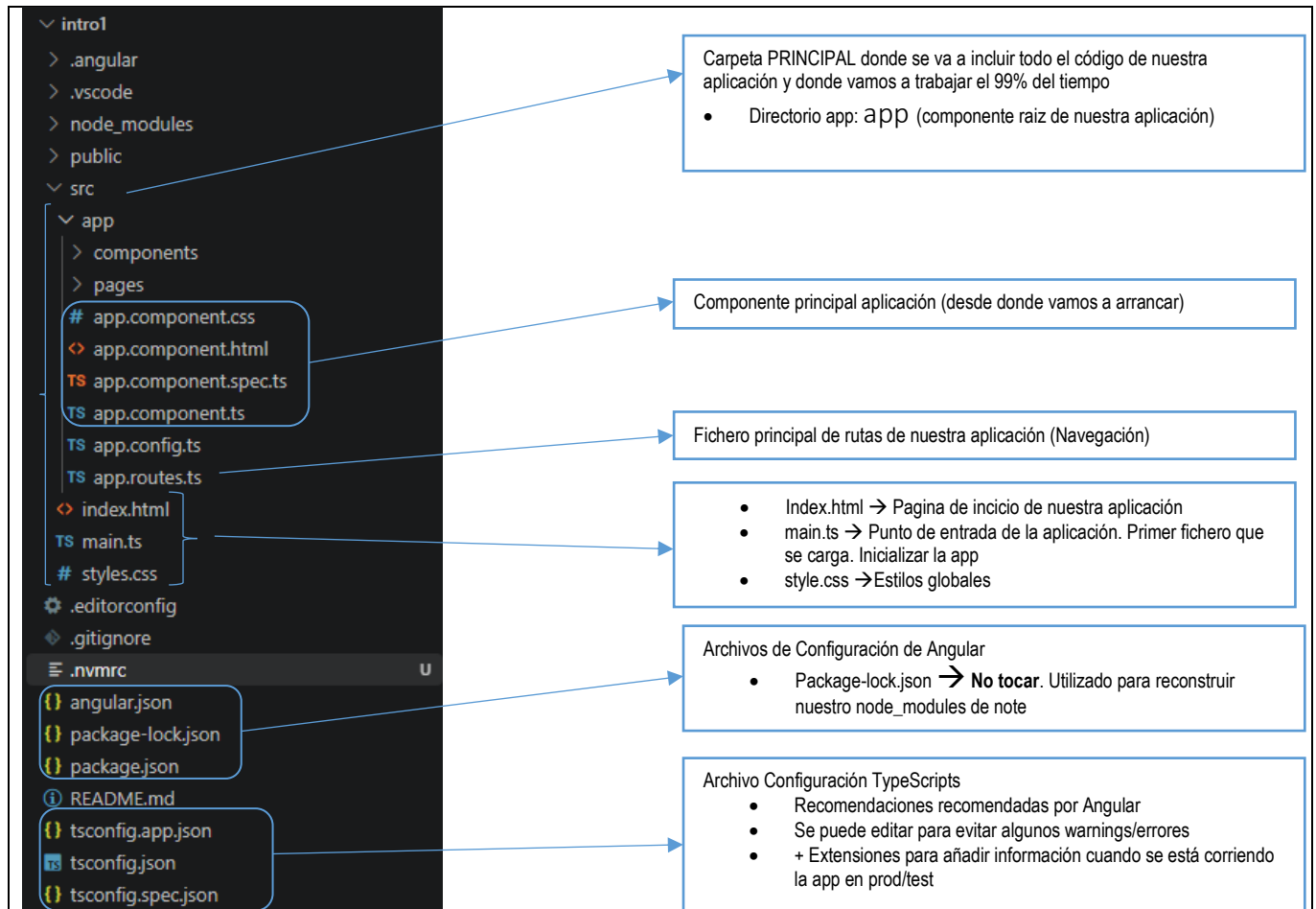
Adicionalmente a este documento, existe otro en la formación del curso con un resumen de los principales comandos de Angular y sus opciones



2.2 Archivos básicos en una aplicación en Angular

Una vez que tenemos nuestro primer proyecto veremos una estructura con un monton de carpetas y archivos. En este apartado vamos a intentar dar una visión general de cada uno de ellos.

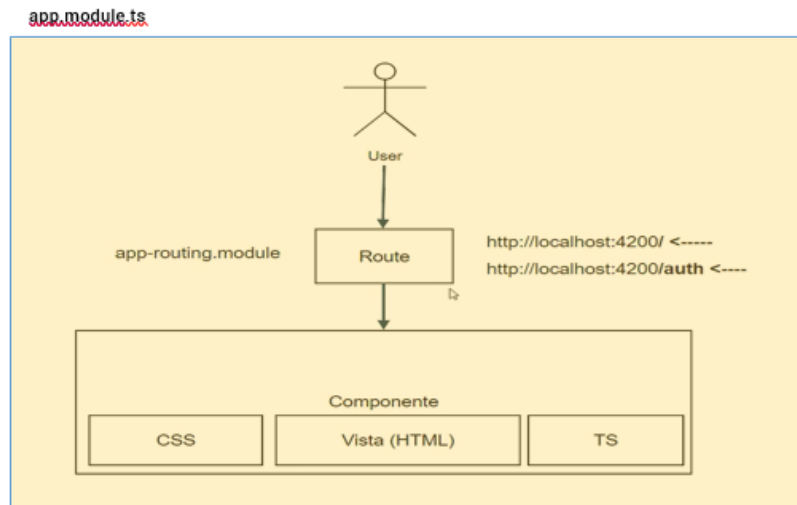
A continuación, se muestran los principales archivos y carpetas en un proyecto Angular



Descripción de otros archivos y carpetas

- `.public`: recursos estaticos de nuestra aplicación (imágenes, iconos, etc)
- `.node_modules`: No tocar. Todos los paquetes referenciados en el `package.json`. Muy pesado pero solo una pequeña parte llega a la implementación final
- `.dist`: compilacion. Lo que nosotros subimos a nuestro servidor. Version de produccion
- `.vscode`: Configuraciones del editor de código de visual studio code
- `.angular`: uso general para manejar cache
- `.gitignore`
- `.editorconfig`: recomendaciones de angular para nuestro código

3 Estructura Base de la aplicación



Esquema del arranque

- Inicia por el `index.html`
- Hay tenemos una referencia al `<app-root>` = (`app.component`) y lo carga en ese punto
- En el `app.component.html` tenemos el `<router-outlet>` que es donde va a renderizar el contenido de la ruta raíz
- Hay mira en el fichero de rutas `app.routes.ts` y cargaría el componente indicado (en este punto vacío)



4 Aplicación Angular: Pensando en componentes

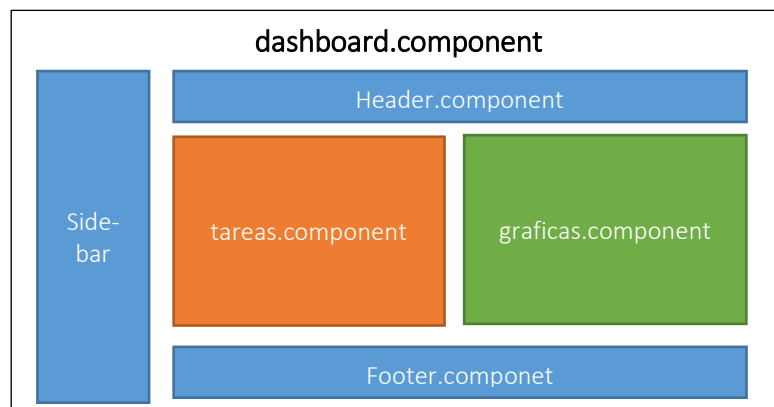
Podemos decir que los compontes son la base de nuestra aplicación.

Un componente se compone básicamente de 3 archivos + 1 opcional

- .html
- .ts
- .css (podría considerarse también opcional)
- .spec.ts (opcional)

¡Concepto importante!:

- Cada página de Angular (será un componente) que está a su vez puede (y debe) estar formada a partir de componentes más pequeños.
- Tenemos que intentar subdividir en componentes lo más pequeños y específicos posibles para mejorar el mantenimiento y reusabilidad



4.1 Estructura base del componente en código

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-header',           //identificador del componente para llamarlo en el HTML
  imports: [RouterLink,RouterLinkActive], //Importaciones requeridas por el componente. Ej. Otros componentes
  templateUrl: './header.component.html', //Fichero de plantilla HTML
  styleUrls: ['./header.component.css']   //Fichero de estilos CSS
})
export class HeaderComponent {
  // aquí irá el código propio con la lógica a implementar de este componente
}
```

4.2 Creación de Componentes

- Comando: `ng generate component </path/nombreComponente> | ng g c </path/nombreComponente>`
- Lo va a generar tomando como base a la carpeta `./app`
- Modificadores opcionales más utilizados

OPCION	DESCRIPCION
<code>--skip-tests (*)</code>	No genera el archivo <code>.spec.ts</code>
<code>--inline-template</code>	No genera el archivo de plantilla <code>.html</code>
<code>--inline-style (*)</code>	No genera el archivo de estilos <code>.css</code>
<code>--dry-run</code>	Muestra lo que generaría sin generarlo



<code>--force</code>	Fuerza a sobrescribir si existe el archivo
----------------------	--

4.3 Componentes Estandar vs Standalone

La diferencia principal entre un componente estándar (basado en módulos) y un componente standalone en Angular radica en cómo gestionan sus dependencias

- Los estándar dependen de un NgModule
- Los standalone son autónomos e importan sus dependencias directamente

Las versiones modernas de Angular están migrando a usar componentes Standalone en lugar de los tradicionales basados por módulos. No obstante, ambos pueden coexistir en un mismo proyecto

Componentes Estándar (Tradicionales con NgModule)

- Declaración: Deben estar declarados obligatoriamente dentro del arreglo `declarations` de un NgModule.
- Dependencias: Para usar otro componente, directiva o pipe, el módulo que los contiene debe importar a su vez el NgModule donde dichos elementos fueron declarados.
- Estructura: Tienden a requerir una arquitectura más pesada con múltiples archivos de módulos (*.module.ts).
- Compartir código: Si quieres reutilizar un componente en otro sitio, debes asegurarte de exportarlo (exports) en su módulo de origen

Componentes Standalone (Autónomos)

- Declaración: No necesitan estar declarados en ningún NgModule. Se configuran con `standalone: true` en su decorador `@Component` (comportamiento por defecto en versiones recientes de Angular).
- Dependencias: Importan otros componentes, directivas o pipes directamente en su propio decorador a través del arreglo `imports`.
- Estructura: Código más limpio, directo y modular, eliminando la necesidad de crear y mantener archivos NgModule intermedios.
- Rendimiento y pruebas: Facilitan la carga diferida (lazy loading) de forma más intuitiva y simplifican las pruebas unitarias al aislar mejor las dependencias

NOTA: El hecho de que un componente sea Standalone influye en su uso y definición en la gestión de módulos en angular que veremos más adelante en este mismo documento

4.3.1 Identificar visualmente componente estándar y standalone

Componentes Estándar (Basados en Módulos)	Componentes standalone (independientes)
○ Contiene la propiedad standalone: false (o carece por completo de la propiedad standalone en versiones anteriores a Angular 19). ○ No tiene una propiedad imports para dependencias locales en su decorador. ○ Debe estar declarado obligatoriamente en la sección <code>declarations</code> de un archivo *.module.ts (NgModule) para poder usarse	○ Contiene la propiedad standalone: true dentro del decorador <code>@Component</code> (en versiones recientes de Angular, es el valor por defecto y puede no estar escrito explícitamente). ○ Incluye una propiedad imports dentro del mismo decorador <code>@Component</code> para cargar otros componentes, directivas o módulos que necesite. ○ No se declara en ningún archivo NgModule
<pre>@Component({ selector: 'app-mi-componente', standalone: false, // Omitido en ver. antiguas })</pre>	<pre>@Component({ selector: 'app-mi-componente', standalone: true, // Omitido en ver. Modernas imports: [CommonModule], })</pre>



4.4 Estructurando la aplicación en directorios / módulos

Antes de empezar a crear componentes tenemos que tener en cuenta que por defecto Angular nos los crea todos justos en la carpeta /app

Por ello es una recomendación muy útil el subdividir en diferentes carpetas para cada uno de nuestros componentes de la aplicación de forma que mantengamos el código estructurado. Cabe mencionar que esto te lo proporciona fácilmente con el uso de módulos de Angular, si bien, tenemos que tener en cuenta que las últimas versiones están reduciendo el uso de los mismos por la utilización de componentes standalone

Una aproximación (sin el uso de módulos que introduciremos más adelante es)

```
\app
  \shared
    \components → componentes con funciones individuales
    \pages       → componentes que conforman una página o vista de nuestra app
    \models      → clases para la definición de tipos de datos, interfaces, etc.
    \services    → servicios
  \clientes      → Carpeta que engloba lo que tiene que ver con gest. clientes
    \components
    \pages
    \models
    \services
  \proveedores   → Carpeta que engloba lo que tiene que ver con gest. proveedores
    \components
    \pages
    \models
    \services
  \...
```



5 Navegación en Angular. Entender el routing

5.1 ¿Qué es el Router?

El **Router** es el encargado de **gestionar la navegación** entre las diferentes vistas o componentes de una aplicación Angular y determina que componente debe mostrarse para cada URL.

No es simplemente un sistema para cambiar URLs: es el mecanismo completo que coordina a través del URL → navegación → guards/resolvers → carga de componentes → renderizado en `<router-outlet>`.

Todo empieza en nuestro componente principal `app.component.html`, donde encontraremos la etiqueta `<router-outlet />`

Aquí es donde se va a renderizar el contenido que tenga que mostrarse en nuestra aplicación dependiendo de la ruta indicada en la URL

- La idea principal es quitar todo el contenido y dejar el `<router-outlet />` para que renderice el contenido que se tienen que mostrar en esa posición

```
<section>
  <router-outlet />
</section>
```

- También tenemos la posibilidad de que si todas las paginas van a contener elementos comunes como header/footer, etc... dejar las referencias a esos objetos en la html. Ejemplo

```
<main>
  <!-- barra de navegación -->
  <nav><navbar-component /></nav>

  <section>
    <router-outlet />
  </section>
</main>
```

5.2 Elementos básicos en el router

Elemento	Función
Router	Gestiona la Navegación
Routes	Definir las rutas disponibles
RouterLink	Permite navegar desde HTML
RouterLinkActive	Marcar la ruta activa
RouterOutlet	Muestra el componente

5.3 ¿Dónde y cómo se definen estas rutas?.

- La respuesta es en archivo **app-routing.module.ts** que se encuentra en el directorio raíz de nuestra aplicación.
- Vamos a definir un array de objetos donde debemos indicar principalmente el path y el componente que tiene que cargarse si existe coincidencia con ese path



```
import { Routes } from '@angular/router';
import { HomeComponent } from '../pages/home/home.component';
import { DashboardComponent } from '../pages/dashboard/dashboard.component';
import { AppComponent } from '../app.component';

export const routes: Routes = [

  { path: '', component: AppComponent },           // --> ruta raíz localhost:4200
  { path: 'home', component: HomeComponent },      // localhost:4200/home
  { path: 'dashboard', component: DashboardComponent },

  // redirección cuando no encuentra ruta
  { path: '**', redirectTo: 'dashboard' } //alternativa es poner un componente de page not found

];
```

5.4 Navegar desde el componente (RouterLink) y mostrar el link activo (RouterLinkActive)

Por último, tenemos que mencionar como podemos gestionar el link a otras páginas de la aplicación, así como con una forma rápida de marcar cual es la ruta activa

- Con **RouterLink** le indicamos al router que navegue a la ruta indicada
- Con **RouterLinkActive** le podemos decir a angular que automáticamente asigne un estilo a nuestro link activo

```
<a [routerLink]="['/contador']" [routerLinkActive]='active">Contador</a>
<a routerLink="/personas" routerLinkActive="active">Prsonas</a>

<!--
• Usa la Opción 1 (Con corchetes []) cuando tu ruta dependa de variables de tu componente.
Ej. Por ejemplo, si el nombre del anime o el ID cambiaran dinámicamente: [routerLink]="['/persona', IdPersoan]
• Usa la Opción 2 (Sin corchetes) cuando la ruta sea completamente estática, fija y escrita a mano en el HTML
, ya que hace que tu código sea más limpio y fácil de leer.
-->
```

Nota: En este ejemplo, angular asignara la clase 'active' al enlace que este activo. Solo tenemos que definir la propiedad css para la clase .active en nuestro fichero de estilos.



6 Ejercicio: Generar componentes y visualizarlo en nuestra aplicación.

Objetivos: Crear componentes básicos y gestionar el router

1. Crear un componente NavBar
 - a. Incluir una serie de links para acceder a nuestras páginas (app-component)
2. Crear un componente Header
 - a. Título de la pagina
3. Crear un componente Ejemplo que incluya los 2 componentes previos y una parte especifica suya
4. Incluirlo el componente en el fichero de rutas
5. Modificar el componente existente header para incluir el link al nuevo componente
6. Verificar el funcionamiento



SECCION 2

7 Dar contenido a nuestros componentes

YA estamos en disposición de empezar a dar contenido real a nuestras páginas de la aplicación. Para ello en esta sección vamos a introducir los siguientes conceptos

- Editar el archivo .html/.css (diseñando la interfaz): Uso de plantillas
- Editar el archivo .ts (datos y funcionalidad)
- Definición de variables, estructuración de datos con interfaces y funciones.
 - Variables y Arrays (miVariable:string="" | myArray: number[]=[1,2,3];)
 - Interfaces (interface: {})
 - Enumerables (enum: {})
- Referenciando a objetos de la clase (this.miVariableDelcomponente)
- Uso de directivas @for, @if y @switch
- Uso de directivas ngClass, ngStyle
- Introducción y uso de Pipe de Angular
- Gestión de eventos en los elementos de nuestra interfaz (evento)="funcionGestion(parámetros)

7.1 Principales directivas en Angular

Una directiva no suele representar una pantalla completa como un componente; sirve para añadir o modificar comportamiento sobre elementos o plantillas.

7.1.1 Directivas Estructurales @for, @if y @switch

Modifican la estructura del DOM, creando o eliminando elementos de la interfaz.

Usos más comunes

- **@for** [sintaxis antigua *ngFor]: Mostrar los datos en la plantilla html recorriendo un array.
- **@if** [sintaxis antigua *ngIf]: Controlar una condición, por ejemplo, para mostrar o no un dato según se cumpla dicha condición
- **@else** (acompaña al @if) para cuando no se cumpla una condición. Puede usarse de forma recursiva
- **@switch** (sintaxis alternativa para implementar una estructura anidada de if/else del tipo if, else if, else if ... else)

Ejemplo:

```
<ul>
  @for (usuario of usuarios; track usuario.id) {
    @if(usuario.edad>=18){
      <li>{{ usuario.nombre }} - {{ usuario.edad }} años </li>
    }
  }
</ul>
```

7.1.2 Directivas de Atributo ngClass, ngStyle

Cambian el comportamiento o estilo de un elemento existente.

- **ngClass**: añade o elimina clases CSS dinámicamente.
- **ngStyle**: modifica estilos directamente.

Ejemplo:

```
<div [ngClass]="{'activo': esActivo, 'texto-rojo': tieneError }">
  Texto con clases dinámicas
```



```
</div>
```

7.1.3 Directiva [ngModel]

ngModel es una directiva de Angular que permite hacer enlace bidireccional de datos (two-way data binding) entre una variable del componente y un elemento HTML, normalmente un formulario.

IMPORTANTE: Para utilizar ngModel, debes importar FormsModule

Ejemplo:

```
<label>
  <input type="checkbox" name="fBajas" value=true [(ngModel)]="filtroBajas"> Filtrar
</label>
```

7.2 Decorador @ViewChild / viewChild() de Angular

En Angular, @ViewChild permite acceder desde el componente TypeScript a un elemento HTML, un componente hijo o una directiva que está dentro de la vista del componente.

Es muy útil cuando necesitas interactuar con un elemento concreto de la plantilla.

Ejemplo: Poner el foco en un elemento de la interfaz al mostrar la pantalla

HTML

```
<input #miInput type="text" placeholder="Escribe tu nombre">
```

TS

```
import {
  Component,
  ViewChild,
  ElementRef,
  AfterViewInit
} from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent implements AfterViewInit {

  @ViewChild('miInput') miInput!: ElementRef<HTMLInputElement>;

  ngAfterViewInit() {
    this.miInput.nativeElement.focus();
  }
}
```

En versiones modernas de Angular también puedes utilizar la función viewChild() en lugar del decorador @ViewChild. Esta API devuelve una señal que contiene la referencia, por lo que se consulta con this.miInput().

NOTA: Se suele utilizar con ngAfterViewInit() [ver ciclo de vida de los componentes en angular] porque Angular necesita inicializar la vista antes de que puedas utilizar de forma segura una referencia a un objeto de la misma.

```
this.miInput.nativeElement.focus()
```



7.3 Uso de estructuras de datos: interfaces, enum y arrays

7.3.1 Uso de interface para la definición de la estructura de un dato complejo

Una interface en Angular es realmente una interfaz de TypeScript. Sirve para definir la estructura y los tipos de datos que debe tener un objeto. Se utiliza principalmente para tipar modelos, objetos, respuestas de APIs, propiedades de componentes y datos de formularios

Ejemplo

```
export interface Usuario {
  id: number;
  nombre: string;
  email: string;
  dni?: string; //al poner ? nos indica que es opcional
}
```

La interface se suele definir en un archivo .ts aparte (ejemplo personas.models.ts) y se exportan e importan en los componentes donde las usemos.

Después las importamos y utilizamos en un componente y TypeScript puede comprobar que el usuario tenga las propiedades y tipos correctos.

```
usuario: Usuario = {
  id: 1,
  nombre: 'Juan',
  email: 'juan@email.com',
  dni: '123445-D'
};
```

Diferencias entre interface vs class

interface	class
Define la estructura de un objeto	Define una estructura y comportamiento
Solo existe durante la comprobación de tipos	Puede existir en tiempo de ejecución
No se instancia con new	Se puede instanciar con new
Muy útil para modelos de datos	Útil cuando necesitas métodos/lógica

7.3.2 Uso de enum para definir una lista de valores enumerables

enum es una característica de TypeScript que permite definir un conjunto de valores constantes con nombre. Es útil cuando una variable solo puede tener determinadas opciones. Nos ayudan a mantener nuestro código más comprensible y evitar errores en las asignaciones de valores.

Ejemplo

```
enum Estado {
  ACTIVO = 'ACTIVO',
  INACTIVO = 'INACTIVO',
  PENDIENTE = 'PENDIENTE'
}
let estado: Estado = Estado.ACTIVO;
```

Idea clave: usa enum cuando tengas un conjunto cerrado de opciones, como ACTIVO/INACTIVO/BAJA

enum vs interface

- interface → define qué propiedades tiene un objeto.



- enum → define qué valores permitidos puede tener algo.

7.3.3 Uso de un array para almacenar un conjunto de datos

En Angular, los arrays son simplemente arrays de JavaScript/TypeScript. Se utilizan constantemente para almacenar listas de datos, por ejemplo usuarios, productos o tareas.

Crear un array (ambas sintaxis son equivalentes)

- nombres: string[] = ['Juan', 'Ana', 'Pedro'];
- nombres: Array<string> = ['Juan', 'Ana', 'Pedro'];

Mostrar un array en el html con la directiva @for

```
<ul>
  @for (usuario of usuarios; track usuario.id) {
    <li>
      {{ usuario.nombre }} - {{ usuario.edad }} años
    </li>
  }
</ul>
```

Principales métodos para el manejo de Arrays

Método	Para qué sirve
push()	Añadir un elemento
pop()	Eliminar el último
shift()	Eliminar el primero
unshift()	Añadir al principio
find()	Buscar un elemento
filter()	Filtrar elementos
map()	Transformar elementos
forEach()	Recorrer elementos
includes()	Comprobar si existe un valor
length	Obtener cantidad de elementos



7.4 Utilidades para la depuración del código.

Tenemos diferentes herramientas que nos pueden ayudar en la depuración de nuestro código

- Comando “console”: Mostrar información en la consola depuración del navegador para ver ciertos valores
 - `Console.log()`
 - `Console.Table(array)`
 - `Console.table(array,[propiedad1,...,propiedad])`

NOTA: Usar para depurar, pero deberemos de recordar quitarlos o comentar para no lanzar mensajes a la consola del navegado que no sean necesarios

- Ejecutar la aplicación en VsCode en modo Depuración [menú: Ejecutar/Depuración]
 - Esto nos permite poder poner break-points en nuestro código, controlar la ejecución y verificar el valor de variables y propiedades
 - Es la opción más útil de depuración.

NOTA: También se disponen de plagios en el navegador para realizar estas operaciones.

7.5 Los Pipe en Angular

En Angular, los **pipes** son una forma de transformar datos directamente en la plantilla HTML, sin modificar el valor original.

Se escriben usando el símbolo |

Ejemplo: `{{ nombre | uppercase }}` → aunque el nombre sea javi se mostrara JAVI

Lista de los principales pipes en Angular

Pipe	Para qué sirve	Ejemplo	Resultado aproximado
uppercase	Convierte a mayúsculas	<code>{{ nombre uppercase }}</code>	JUAN PÉREZ
lowercase	Convierte a minúsculas	<code>{{ nombre lowercase }}</code>	juan pérez
titlecase	Primera letra de cada palabra en mayúscula	<code>{{ nombre titlecase }}</code>	Juan Pérez
date	Formatea fechas	<code>{{ fecha date:'dd/MM/yyyy' }}</code>	05/10/2026
currency	Formatea monedas	<code>{{ precio currency:'EUR' }}</code>	49,99 €
number	Formatea números	<code>{{ precio number:'1.2-2' }}</code>	49,99
percent	Convierte a porcentaje	<code>{{ descuento percent }}</code>	25%
json	Convierte un objeto a JSON	<code>{{ usuario json }}</code>	<code>{"nombre":"Juan"}</code>
slice	Obtiene una parte de un array/string	<code>{{ nombres slice:0:3 }}</code>	Primeros 3 elementos
async	Obtiene el valor de un Observable/Promise	<code>{{ usuarios\$ async }}</code>	Valor emitido
keyvalue	Convierte un objeto/mapa en pares clave-valor	<code>{{ usuario keyvalue }}</code>	nombre → Juan
i18nSelect	Selecciona un texto según un valor	<code>{{ genero i18nSelect:mensajes }}</code>	Texto correspondiente
i18nPlural	Texto singular/plural según cantidad	<code>{{ cantidad i18nPlural:mensajes }}</code>	1 usuario / 5 usuarios



7.6 Ejercicio: Crear una pantalla con un listado de clientes

1. Crear el componente (listaClientes).
2. Meterlo en el router.ts
3. Modificar nuestro componente NavBar
4. Probar que se accede directamente con la barra de navegación
5. Incorporarlo al nav_bar componente (añadir una entrada al array)
6. Debería salir visualmente y poder navegar
7. Implementar el componente (listaClientes)
 - ** archivo .ts
 - Definirme una interface
 - Definirme un array del tipo de esa interfaz y asignarle valores
 - ** archivo.html (Pasos para el uso del @for en las pantallas)
 - Copiar la plantilla html y el fichero de estilos css
 - uso del @for (item of listaItem; track item.ID)
 - Debemos sustituir el contenido estático de la plantilla por el uso de un @for

7.7 Ejercicio 2: Modificar pantalla con un listado de clientes

1. Modificar la interfaz para meter propiedad estado de tipo enumerable
2. Incluir valor de la nueva propiedad en la tabla
3. Modificar el color de como se muestra el valor del estado con el uso de ngClass o versión alternativa con @
4. Gestionar evento (click) en las filas de la tabla para seleccionar la empresa
 - a. Propiedad para asignar el valor correspondiente
 - b. Mostrar info en el log
 - c. Incluir nueva área para mostrarlo en pantalla
5. Incluir un check (formulario) para implementar un filtro de ocultar bajas
 - a. Uso de ngModel
 - b. Uso de @if para no mostrar la fila si esta de baja

7.8 Cuadro resumen interacción con el html

Sintaxis en el HTML	Función
{{ nombre }}	Mostrar un valor de la variable nombre en el html
[value]="nombre"	Pasar datos al HTML. Asignar el valor de nombre
(click)="guardar()"	Capturar un evento y llamar a la función de gestión en el .ts
[(ngModel)]="nombre"	Leer y modificar el valor (enlace bidireccional)



7.9 Introducción al uso de señales

- Definición de las señales vs propiedades
- Ejercicio página contador
- Definir propiedades vs señales
 - Definición: Uso de () para indicar que es una señal
- Definir una función
 - Modificar valores: asignación vs métodos de las señales (set/update)
- Ejercicio: Repicar he incluir un botón de suma



SECCION 3: Servicios en Angular

8 Que son los servicios

Un **servicio en Angular** es una clase de TypeScript con un propósito específico que encapsula lógica de negocio, datos o funciones reutilizables fuera de los componentes visuales

8.1.1 ¿Para qué sirven?

- **Separar responsabilidades:** Saca el código pesado o repetitivo de los componentes para mantenerlos limpios y enfocados solo en la interfaz de usuario.
- **Compartir datos:** Permite sincronizar información y estado entre diferentes componentes que no están directamente conectados.
- **Conectar con APIs:** Gestiona las peticiones HTTP (como GET o POST) hacia un servidor o backend externo.
- **Reutilizar código:** Evita duplicar funciones idénticas en varias partes de la aplicación

8.1.2 Características Especiales

- **Patrón Singleton:** Por defecto, Angular crea una sola instancia del servicio para toda la aplicación, lo que permite conservar el estado de los datos al navegar entre pantallas.
- **Inyección de dependencias (DI):** Angular se encarga de entregar e instanciar el servicio automáticamente donde se solicite mediante el decorador `@Injectable()` y la función `inject()`

8.2 Creando un servicio

- Comando: **ng generate service /path/nombreServicio | ng g s /path/nombreServicio**
Ejemplo: `ng g s /modules/clientes/services/cliente --skip-tests`
- Modificadores opcionales de utilidad.

OPCION	DESCRIPCION
<code>--skip-tests (*)</code>	No genera el archivo <code>.spect.ts</code>

8.3 Trabajar con el servicio creado

Para poder hacer uso de un servicio lo primero que tenemos que hacer es 'Inyectar' el servicio en los componentes que vayamos a utilizarlo. Esto se hace a través del decorador `@Injectable` (es como si importáramos la instancia del servidor, pero solo se crea si es la primera vez que se llama)



8.4 Ejercicio: Servicio Básico

Objetivo: Crear un servicio básico para traspasar nuestra lógica de gestión de lista de clientes al mismo

1. Crear el servicio
2. Incluir las importaciones de las interfaces y otros tipos de datos a utilizar
3. Definir nuestro array de datos internamente dentro del servicio
4. Traspasar los métodos de gestión del componente al servicio
5. Inyectar el servicio para usarlo en nuestro componente
 - `private servicioClientes = inject(ClientesService);`
6. Hacer uso del servicio y los métodos publicados en nuestro componente
 - Limpiar datos innecesarios en nuestro componente
7. Verificar funcionamiento.

Sugerencia: mantener el componente inicial y hacer una copia.

- Crear nuevo componente
- Modificar router
- Aplicar los cambios sobre el nuevo componente.



9 Uso de Servicios para el consumo de APIs por peticiones HTTP

9.1 Ejemplo práctico de cómo usar un servicio conectado con el consumo de una API

9.1.1 Paso 1: Configurar el Proveedor HTTP

Antes de usar las peticiones HTTP, debes habilitar el cliente en la configuración global de tu aplicación (normalmente en `app.config.ts`).

Fichero de configuración de nuestra aplicación (`app.config.ts`)

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http'; // 1. Importar el proveedor

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient() // 2. Agregar a la lista de proveedores
  ]
};
```

9.1.2 Paso 2: Crear el Servicio (`user.service.ts`)

Este servicio se encargará de hacer la petición **GET** a la API y devolver un Observable con los datos.

Fichero typescript de nuestro servicio user

```
// user.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root' // Hace que el servicio sea Singleton y global
})
export class UserService {
  // 1. Inyectar el cliente HTTP de Angular
  private http = inject(HttpClient);

  // URL de la API de pruebas
  private apiUrl = 'https://typicode.com';

  // 2. Método para obtener los usuarios
  getUsers(): Observable<any[]> {
    return this.http.get<any[]>(this.apiUrl);
  }
}
```

9.1.3 Paso 3: Consumir el Servicio en el Componente (`user.component.ts`)

Injectamos el servicio en el componente, nos suscribimos al método y guardamos el resultado en una variable.

Fichero typescript de nuestro componente user

```
// user.component.ts
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { UserService } from '../user.service'; // Importar el servicio

@Component({
  selector: 'app-user',
  standalone: true,
  imports: [CommonModule], // Necesario para usar la directiva *ngFor
  templateUrl: '../user.component.html'
})
```



```
})
export class UserComponent implements OnInit {
  // 1. Inyectar el servicio creado
  private userService = inject(UserService);

  // Variable para almacenar la lista de usuarios
  usuarios: any[] = [];

  ngOnInit(): void {
    // 2. Llamar al servicio al iniciar el componente
    this.userService getUsers().subscribe({
      next: (data) => {
        this.usuarios = data; // Guardar los datos recibidos
      },
      error: (err) => {
        console.error('Error al traer los usuarios:', err);
      }
    });
  }
}
```

9.1.4 Paso 4: Mostrar los Datos en el HTML (`user.component.html`)

Recorremos el arreglo de usuarios usando las nuevas estructuras de control de flujo de Angular o la directiva clásica.

Fichero html de nuestros componente user

```
<!-- user.component.html -->
<h2>Lista de Usuarios desde la API</h2>

<ul>
  @for (usuario of usuarios; track usuario.id) {
    <li>
      <strong>{{ usuario.name }}</strong> - {{ usuario.email }}
    </li>
  } @empty {
    <li>Cargando usuarios o lista vacía...</li>
  }
</ul>
```

IMPORTANTE: Estructura de uso al trabajar con los observable

Un Observable es una forma de representar datos que llegarán en el futuro.

```
servicio()
  .subscribe({
    next: (datos) => {
      // Aquí ya tengo los datos
    },
    error: (error) => {
      // Aquí ha ocurrido un error
    },
    complete: () => {
      // Observable terminado
    },
  });

o next = resultado correcto
o error = resultado incorrecto
```



SECCION 4: Comunicación entre componentes

10 Comunicación entre componentes

La comunicación entre componentes nos permite pasar información entre el componente padre y sus componentes los hijos y viceversa.

Para ello vamos a usar los decoradores de Angular `input()/@input` y `output()/@output`

Dirección	API tradicional	API moderna
Padre → Hijo	@Input()	input()
Hijo → Padre	@Output()	output()

10.1 Comunicación con componente hijo (input)

El objetivo es el paso de información del componente padre al componente hijo
Usamos input

10.1.1 Ejemplo con decorador tradicional @Input()

Imagina un componente padre que envía el nombre de un usuario a un componente hijo.

Componente hijo:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<p>Hola, {{ name }}</p>`
})
export class UserComponent {
  @Input() name: string = '';
}
```

Componente padre:

```
@Component({
  selector: 'app-parent',
  template: `<app-user [name]="Carlos" />`
})
export class ParentComponent {}
```

Aquí name es una propiedad normal de TypeScript. Angular actualiza su valor cuando cambia el binding del padre.

10.1.2 Ejemplo con nuevo decorador input

Ahora hacemos exactamente lo mismo, pero utilizando signals.

Componente hijo:

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-user',
  template: `<p>Hola, {{ name() }}</p>`
})
export class UserComponent {
  name = input<string>('');
}
```

El componente padre no necesita cambiar:

```
<app-user [name]="Carlos" />
```



La diferencia clave es que `name` ahora es un signal de solo lectura. Por eso, para leerlo, se utiliza `name()`.

10.1.3 Inputs obligatorios

Ambas APIs permiten declarar inputs obligatorios.

- Con `@Input():` → `@Input({ required: true }) name!: string;`
- Con `input():` → `name = input.required<string>();`

En ambos casos, Angular puede detectar durante la compilación si el componente se utiliza sin proporcionar ese input.

10.1.4 Reactividad: la diferencia más importante

Aquí es donde `input()` destaca especialmente.

Con `@Input()`, si quieres reaccionar a cambios de una propiedad, puedes utilizar `ngOnChanges`.

```
import {
  Component,
  Input,
  OnChanges,
  SimpleChanges
} from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `{{ count }}`
})
export class CounterComponent implements OnChanges {
  @Input() count = 0;

  ngOnChanges(changes: SimpleChanges) {
    if (changes['count']) {
      console.log('Nuevo valor:', this.count);
    }
  }
}
```

Con `input()`, puedes combinar el input con `computed()` o `effect()`.

```
import {
  Component,
  input,
  computed,
  effect
} from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <p>Valor: {{ count() }}</p>
    <p>Doble: {{ doubleCount() }}</p>
  `
})
export class CounterComponent {
  count = input(0);

  doubleCount = computed(() => this.count() * 2);

  constructor() {
    effect(() => {
      console.log('Nuevo valor:', this.count());
    });
  }
}
```

En este caso:

- `count()` obtiene el valor actual del input.
- `computed()` calcula automáticamente el doble cuando cambia `count`.



- `effect()` ejecuta su función cuando cambia el signal del que depende.

Esto permite trabajar de forma más declarativa con la reactividad de Angular.

Recomendaciones de uso y resumen final:

- Para componentes nuevos en Angular 19 o superior, utiliza `input()` por su integración directa con signals y su enfoque declarativo.
- No obstante, `@Input()` sigue siendo totalmente válido y no necesitas migrar todo un proyecto existente únicamente por esta diferencia.
- Un detalle importante: `input()` crea un signal de solo lectura desde el componente que lo recibe. No debes intentar actualizarlo directamente con `this.name.set(...)`. Si necesitas comunicar cambios al padre, puedes utilizar `output()`, `model()` cuando corresponda o eventos específicos.
- En resumen:
 - `@Input()` es una propiedad que Angular actualiza;
 - `input()` es un signal que Angular actualiza y del que puedes derivar otra lógica reactiva.

10.2 Comunicación con el componente padre (output)

En este caso el objetivo es el paso de información del componente hijo al componente padre. Utilizaremos `output()` API moderna de Angular, o `@Output()` + `EventEmitter` en la versión clásica

10.2.1 Forma clásica @Output

Componente hijo:

```
import { Component, EventEmitter, Output } from '@angular/core';

@Component({
  selector: 'app-hijo',
  template: `
    <button (click)="seleccionarProducto()">
      Seleccionar
    </button>
  `
})
export class HijoComponent {

  @Output()
  productoSeleccionado = new EventEmitter<number>();

  seleccionarProducto() {
    this.productoSeleccionado.emit(123);
  }
}
```

Componente Padre:

```
<app-hijo
  (productoSeleccionado)="recibirProducto($event)">
</app-hijo>
recibirProducto(id: number) {
  console.log('Producto seleccionado:', id);
}
```

10.2.2 ¿Qué es exactamente \$event?

`$event` representa **el valor que el hijo ha emitido**.

Cuando hacemos:



```
this.productoSeleccionado.emit(123);
```

Angular pasa ese 123 al padre.

Por eso:

```
<app-hijo
  (productoSeleccionado)="recibirProducto($event)">
</app-hijo>
```

es equivalente conceptualmente a:

```
recibirProducto(123);
```

10.2.3 La nueva API output()

En Angular moderno podemos hacerlo de una forma más sencilla:

Componente hijo:

```
import { Component, output } from '@angular/core';

@Component({
  selector: 'app-hijo',
  template: `
    <button (click)="seleccionarProducto()">
      Seleccionar
    </button>
  `
})
export class HijoComponent {

  productoSeleccionado = output<number>();

  seleccionarProducto() {
    this.productoSeleccionado.emit(123);
  }
}
```

Y el padre **no cambia**:

```
<app-hijo
  (productoSeleccionado)="recibirProducto($event)">
</app-hijo>
recibirProducto(id: number) {
  console.log(id);
}
```

La diferencia importante está en la declaración:

- API antigua → `@Output() productoSeleccionado = new EventEmitter<number>();`
- API moderna → `productoSeleccionado = output<number>();`

10.2.4 Parámetros que podemos pasar

- No estás limitado a pasar un número o un string.
- Puedes emitir un objeto

```
(empresaSeleccionada)=SeleccionarEmpresa($event)
```

- También puedes emitir un evento vacío o null

```
cerrar() {
  this.cerrado.emit();
}
<app-hijo
  (cerrado)="cerrarModal()">
  // Aquí no existe $event porque no estamos emitiendo ningún valor
</app-hijo>
```

IMPORTANTE

Una cosa MUY importante: `output()` no es para "pasar parámetros" directamente, Esto suele generar confusión.



El valor sale del hijo y llega al padre como \$event

Para que el hijo "reciba" el 123.

No hacemos:

```
<app-hijo  
  (productoSeleccionado)="recibirProducto(123)">  
</app-hijo>
```

La dirección es:

```
      output ()  
Hijo  -----> Padre  
      emit(valor)
```

Por ejemplo:

```
// HIJO  
productoSeleccionado = output<number>();  
  
this.productoSeleccionado.emit(123);
```

y:

```
<!-- PADRE -->  
<app-hijo  
  (productoSeleccionado)="recibirProducto($event)">  
</app-hijo>
```

El 123 sale del hijo y llega al padre como \$event.



10.3 Ejercicio: Pantalla de listado de clientes

Extrae del componente lista-clientes.component el área donde se marca la empresa seleccionada y el filtro
Colocar en el componente padre (page-lista-clientes.component)
Comunicar componentes padre e hijo

Objetivo: Obtener del hijo la empresa seleccionada y pasar del padre al hijo la opción de filtrar bajas

- Salida del hijo de la empresa seleccionada
- Recepción en el hijo del valor de si deben filtrar las bajas

Claves

1. Componente hijo (lista-cliente-final.component.ts)

```
import { output } from '@angular/core';  
outEmpresaSeleccionada = output<Empresa>();
```

Cuando selecciones la empresa:

```
seleccionarEmpresa(empresa: Empresa) {  
  this.outEmpresaSeleccionada.emit(empresa);  
}
```

2. Componente padre (page-cliente-final.component.ts)

• HTML

```
<app-lista-cliente-final  
  (outEmpresaSeleccionada)="outEmpresaSeleccionada($event)">  
</app-lista-cliente-final>
```

• TS

```
empresaSeleccionada!: Empresa;  
  
outEmpresaSeleccionada(empresa: Empresa) {  
  this.empresaSeleccionada = empresa;  
}
```

3. Por tanto, el flujo es:

```
Hijo  
↓  
this.outEmpresaSeleccionada.emit(empresa)  
↓  
$event  
↓  
outEmpresaSeleccionada($event)  
↓  
this.empresaSeleccionada = empresa
```

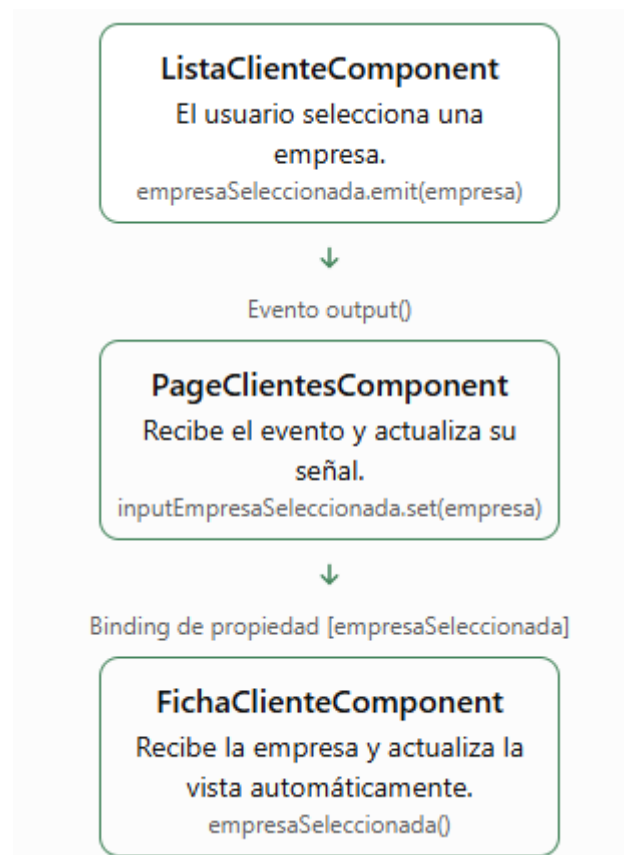
10.4 Ejemplo 2

Crear una nueva página de clientes con 2 componentes hijos

- Lista-clientes (listado de clientes)
- Ficha-cliente (ficha con los datos del cliente seleccionado)

Objetivo: Comunicación entre ellas a través del componente padre

Esquema de cómo debe funcionar la función la comunicación



Mejora: Implementar en la ficha una función para modificar o insertar valores y estos se traspasen a la lista de clientes. (comunicación bidireccional entre los dos componentes hijos)

SECCION 5: Módulos en Angular

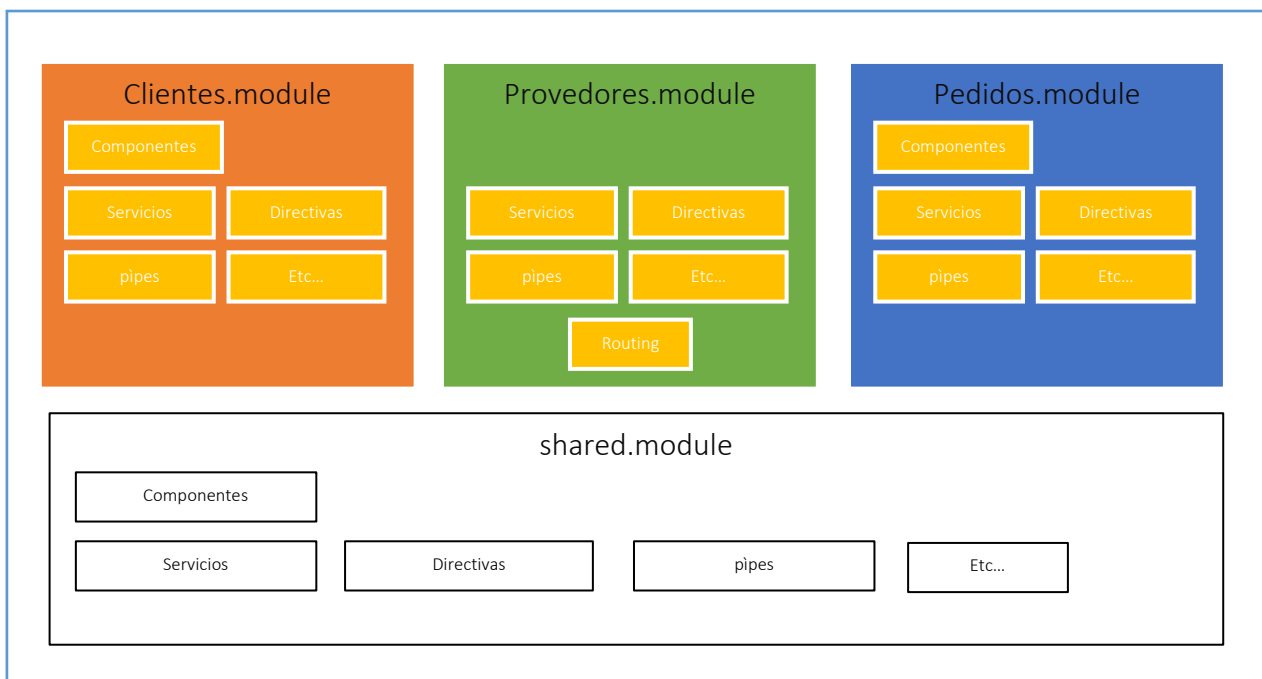
11 Que son los módulos en Angular

Los módulos en Angular se identifican por el uso del Decorador @NgModule y son contenedores lógicos que sirven para agrupar componentes, directivas, pipes y servicios relacionados entre sí.

- Organizan la aplicación: Subdividiendo la aplicación en bloques con funcionalidades relacionadas, más pequeños, ordenados y fáciles de mantener.
- Facilitan la carga bajo demanda (lazyLoading)

1.1 Esquema organizativo mediante módulos.

app.module.ts



NOTA

En las versiones recientes de Angular, el estándar actual son los **componentes standalone** (independientes), los cuales permiten declarar dependencias directamente en cada componente sin necesidad de crear un archivo de módulos.

Estructura de un modulo (@NgModule)

- **Declarations:** Componentes, directivas y pipes que pertenecen a este módulo.
- **Imports:** Otros módulos necesarios para que este funcione
- **Providers:** Servicios que estarán disponibles globalmente en el ámbito del módulo.
- **Exports:** Elementos de este módulo que otros módulos pueden usar.



Ejemplo código de un modulo

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ClientesRoutingModule } from './clientes-routing.module';
import { FichaClienteComponent } from './pages/ficha-cliente/ficha-cliente.component';
import { ListaClientesComponent } from './pages/lista-clientes/lista-clientes.component';
import { Router } from '@angular/router';

@NgModule({
  declarations: [
    // Componentes que contiene el Modulo (ejemplos)
    // FichaClienteComponent,
    // ListaClientesComponent
  ],
  imports: [
    CommonModule,
    // Importaciones de otros Modulos de la aplicacion

    // Si usamos componentes standalone los deberiamos incluir directamente aqui para poder
    exportarlos
    FichaClienteComponent,
    ListaClientesComponent
  ],
  exports: [
    FichaClienteComponent,
    ListaClientesComponent,
  ]
})
export class ClientesModule { }
```

11.1 Creación de los Modulos

- Comando: **ng generate modulo /path/nombreModulo | ng g g /path/nombreModulo**
Ejemplo: **ng g m \modules\clientes --routing**
- Modificadores opcionales de utilidad.

OPCION	DESCRIPCION
--routing	Generar adicionalmente un fichero de rutas propio del modulo → Gestion de lazyLoading y de rutas hijas

11.2 Comunicación entre Módulos

Tenemos que quedarnos con la idea de que para poder usar componentes de un módulo en otros módulos debemos

1. Declararlos en el módulo origen
2. Exportarlos desde el modulo origen
3. Importarlos en el módulo o componente donde se quiera usar

11.3 Como organizar el contenido de un modulo

- Como hemos visto anteriormente, el modulo es como un contenedor que engloba del resto de objetos y clases relacionados conceptualmente entre sí.
- La idea es que una vez que tenemos el creado el modulo, vamos a rellenarlo de componentes, servicios, etc.
- Una estructura recomendada para organizar esta información dentro del módulo podría ser:



Ejemplo: Modulo usuarios:

```
usuarios/
├── components/      # Componentes visuales específicos de este módulo
│   ├── usuario-list/
│   └── usuario-detail/
├── pages/           # Componentes contenedores o vistas principales (Smart Components)
│   └── usuarios-page/
├── services/        # Servicios de datos específicos de este módulo
│   └── usuario.service.ts
├── models/          # Interfaces y tipos de datos del módulo
│   └── usuario.model.ts
├── usuarios-routing.module.ts # Rutas hijas del módulo (lazy loading)
└── usuarios.module.ts    # Definición del @NgModule (si aplica)
```



11.4 Ejercicio práctico:

1. Crear nuestro módulo de clientes con fichero de rutas
2. Crear estructura de carpetas básicas
3. Mover y/o crear nuestros componentes en una estructura de módulos.
4. Crear componente page-ficha-cliente
5. Crear componente page-lista-clientes



SECCION 6

12 Navegación en Angular (II). Ampliación del uso de rutas.

En este apartado vamos a seguir profundizando en el uso de rutas incluyendo nuevas posibilidades más avanzadas.

12.1 LazyLoading y rutas hijas

La idea del LazyLoading es cargar los diferentes recursos solo si van a ser usados y no todos inicialmente. De esta forma podemos mejorar el rendimiento y respuesta de nuestra aplicación al iniciarse.

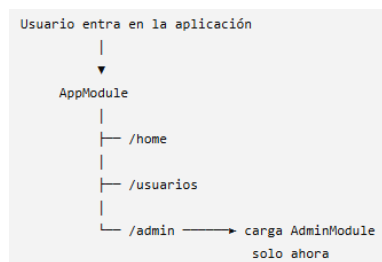
NOTA: Lazy loading = dividir la aplicación en partes y descargar cada parte únicamente cuando el usuario la necesita.

12.1.1 Angular tradicional: Uso con Módulos

Usar **lazyloading** con módulos significa que un módulo de la aplicación no se descarga al cargar inicialmente la aplicación, sino que se descarga solo cuando el usuario navega a una ruta que lo necesita.

La idea para su implementación es:

Supongamos un módulo de administración (AdminModule) que se carga solo cuando entramos en /admin.



1. Definir rutas hijas en el fichero de rutas específico de nuestros módulos

- Para ello debemos haber creado el módulo con rutas y tenerlo declarado para su uso desde el exterior
- Dentro del fichero de rutas del módulo definimos las rutas relativas a ese módulo

```
const routes: Routes = [
  {
    path: '',
    component: AdminComponent
  },
  {
    path: 'usuarios',
    component: UsuariosComponent
  }
];
```

2. Carga dinámica en el Módulo principal de rutas (app-routing.module.ts)

- Definimos inicialmente las rutas de los componentes principales.
- Cargamos bajo demanda. Importante: La carga se realiza como una promesa

```
const routes: Routes = [
  {
    path: 'admin', loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
  }
];
```

El enrutamiento nos quedaría

```
/          → aplicación principal
/admin     → carga AdminModule
```



```
/admin/usuarios → UsuariosComponent
```

12.1.2 Angular moderno: Uso con componentes standalone

En Angular moderno nos encontramos con los componentes standalone, donde ya no necesitas NgModule:

```
{
  path: 'admin',
  loadComponent: () =>
    import('./admin/admin.component')
    .then(m => m.AdminComponent)
}
```

12.2 Paso de parámetros en las rutas

El uso de parámetros en las rutas de Angular permite enviar información dinámica a través de la URL (como un ID) para reutilizar un mismo componente

1. **Para definir** e indicar el uso de parámetros utilizaremos dos puntos (:) seguido del nombre del parámetro en la propiedad path de nuestras rutas configuradas en aap.routes.ts

```
{ path: 'articulos/:id', component: ArticuloDetalleComponent }
```

NOTA: los parámetros de URL son strings, por lo que si necesitas un número tendrás que convertirlo

Si quieres que una misma ruta admita o no parámetros una forma fácil es crear 2 entradas en el router. Por Ejemplo:

```
{ path: 'articulos', component: ArticuloDetalleComponent }
{ path: 'articulos/:id', component: ArticuloDetalleComponent }
```

2. **Para obtenerlo** en tu componente debes usar ActivatedRoute (importarlo)

```
import { Component, inject } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-usuario',
  template: `<p>Usuario: {{ id }}</p>`
})
export class UsuarioComponent {

  private route = inject(ActivatedRoute);

  id = this.route.snapshot.paramMap.get('id');
}
```

3. Para utilizarlo en la **navegación**. Necesitas inyectar el Router

- TS:

```
private router = inject(Router);
this.router.navigate(['/usuarios', 25]);
```
- HTML:

```
<a [routerLink]="['/usuarios', usuario.id]"> Ver usuario </a>
```

SECCION 7: Ciclo de vida de los componentes

13 Ciclo de vida de un componente Angular

El ciclo de vida de un componente describe las distintas etapas por las que pasa desde que se crea hasta que se destruye.

El orden habitual de los principales eventos que se generan en la vida de un componente son:

```
constructor()
↓
ngOnChanges()
↓
ngOnInit()
↓
ngDoCheck()
↓
ngAfterContentInit()
↓
ngAfterContentChecked()
↓
ngAfterViewInit()
↓
ngAfterViewChecked()
↓
... actualizaciones ...
↓
ngOnDestroy()
```

Detallamos a continuación el significado y uso principal de cada uno de ellos

1. constructor()

- Se ejecuta cuando Angular crea la instancia de la clase.
- Se utiliza principalmente para **inyección de dependencias**:

```
constructor(private userService: UserService) {}
```

- No es recomendable utilizarlo para lógica que dependa de que Angular haya inicializado el componente.

2. ngOnChanges()

- Se ejecuta cuando cambia un dato recibido mediante `@Input()`.

```
@Input() user!: User;
ngOnChanges(changes: SimpleChanges) {console.log(changes); }
```

Por ejemplo:

```
<app-user [user]="selectedUser"></app-user>
```

Si `selectedUser` cambia, Angular puede ejecutar `ngOnChanges()`.

- **Importante:** *se ejecuta antes de `ngOnInit()` si existen `@Input()` iniciales.*

3. ngOnInit()

- Se ejecuta **una sola vez**, después de que Angular haya inicializado los `@Input()`.
- Es uno de los hooks más utilizados:



```
ngOnInit() { this.loadUsers(); }
```

- Es apropiado para:
 - Cargar datos.
 - Inicializar variables.
 - Realizar peticiones HTTP.
 - Configurar lógica inicial.

4. ngDoCheck()

- Permite detectar manualmente cambios que Angular no detectaría mediante los mecanismos normales.

```
ngDoCheck() {  
    console.log('Comprobación de cambios');  
}
```

- Importante: Se ejecuta muchas veces, por lo que no conviene poner operaciones pesadas aquí.

5. ngAfterContentInit()

- Se ejecuta una vez después de que Angular haya proyectado contenido mediante `ng-content`.

```
<ng-content></ng-content>
```

- Es útil cuando trabajamos con **content projection**.

6. ngAfterContentChecked()

- Se ejecuta después de que Angular compruebe el contenido proyectado.

```
ngAfterContentChecked() {  
    // ...  
}
```

- Importante: Al igual que `ngDoCheck()`, puede ejecutarse muchas veces.

7. ngAfterViewInit()

- Se ejecuta una vez después de que Angular haya inicializado la **vista del componente** y sus vistas hijas.
- Por ejemplo, si tenemos:

```
<input #nameInput>  
  
podemos acceder a él mediante:  
  
@ViewChild('nameInput') nameInput!: ElementRef;  
  
ngAfterViewInit() {  
    this.nameInput.nativeElement.focus();  
}
```

- Es especialmente útil para trabajar con elementos de la vista (html) mediante `@ViewChild`.

8. ngAfterViewChecked()

- Se ejecuta después de que Angular haya comprobado la vista del componente.

```
ngAfterViewChecked() {  
    // ...  
}
```



```
}
```

- ***Importante:** Puede ejecutarse muchas veces, por lo que hay que utilizarlo con cuidado.*

9. ngOnDestroy()

- Se ejecuta justo antes de que Angular destruya el componente.

```
ngOnDestroy() {  
    console.log('Componente destruido');  
}
```

- Tradicionalmente se utiliza para limpiar:
 - Suscripciones.
 - Timers.
 - Listeners.
 - Recursos externos.

Por ejemplo:

```
ngOnDestroy() {  
    this.subscription.unsubscribe();  
}
```

13.1.1 Tabla Resumen

Hook	¿Cuándo salta?	Uso principal
Constructor (*)	Al crear la instancia	Inyección de dependencias
ngOnChanges (*)	Cambia un @Input	Reaccionar a cambios de entrada
ngOnInit (*)	Una vez al inicializar	Inicialización y carga de datos
ngDoCheck	En comprobaciones	Detección personalizada
ngAfterContentInit	Contenido proyectado inicializado	ng-content
ngAfterContentChecked	Se comprueba contenido	Casos avanzados
ngAfterViewInit	Vista inicializada	@ViewChild, DOM
ngAfterViewChecked	Se comprueba la vista	Casos avanzados
ngOnDestroy (*)	Antes de destruir	Limpieza

() Los más importantes y/o utilizados*



SECCION 8: Formularios en Angular

En Angular existen principalmente dos formas de trabajar con formularios:

- **Forms dirigidos por plantilla (Template-driven Forms):** gran parte de la lógica está en el HTML.
- **Formularios reactivos (Reactive Forms):** el formulario se define en TypeScript mediante objetos como FormControl, FormGroup y FormArray.

NOTA: También podríamos crear un formulario básico con etiquetas html y llevar toda la logica y control al typeScript

14 Formularios Basados en Plantillas (ngForm)

NgForm se utiliza para crear formularios basados en plantillas (Template-driven forms) en Angular.

Permite gestionar valores, estado y validaciones de un formulario directamente desde la plantilla, siendo especialmente cómodo para formularios sencillos.

IMPORTANTE: Para su utilización debemos importar **FormsModule, NgForm**.

14.1 Conceptos principales

- **#formulario="ngForm"**: crea una referencia al formulario gestionado por Angular.
- **ngModel**: conecta el campo HTML con una propiedad del componente.
- **name**: es necesario para que Angular registre el campo dentro de NgForm.
- **required**: añade una validación obligatoria.
- **formulario.value**: contiene los valores introducidos.
- **formulario.valid / formulario.invalid**: indican si el formulario cumple las validaciones.
- **(ngSubmit)**: ejecuta una función al enviar el formulario.

14.2 Ejemplo básico

HTML

```
<form #formulario="ngForm" (ngSubmit)="enviar(formulario)">
  <input
    type="text"
    name="nombre"
    [(ngModel)]="nombre"
    required
  />

  <button type="submit" [disabled]="formulario.invalid">
    Enviar
  </button>
</form>
```

TS:

```
import { Component } from '@angular/core';
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  selector: 'app-formulario',
  imports: [FormsModule],
  templateUrl: './formulario.html'
})
```



```
export class FormularioComponent {  
  nombre = '';  
  
  enviar(formulario: NgForm) {  
    console.log(formulario.value);  
  }  
}
```

15 Formularios Reactivos en Angular

Los **formularios reactivos (Reactive Forms)** de Angular permiten crear y gestionar formularios de manera programática, manteniendo el estado y las validaciones del formulario en el código TypeScript.

Los formularios reactivos son especialmente útiles cuando tenemos formularios complejos, muchas validaciones o campos dinámicos.

IMPORTANTE: Para su utilización debemos importar **ReactiveFormsModule**:

15.1 Conceptos principales

- **FormGroup**: agrupa y gestiona varios controles del formulario.
- **FormControl**: representa un campo individual.
- **formControlName**: conecta un campo HTML con un FormControl.
- **[formGroup]**: conecta el formulario HTML con el FormGroup.
- **Validators**: permite añadir validaciones como required, email, minLength, etc.
- **formulario.value**: contiene los valores introducidos.
- **formulario.valid / formulario.invalid**: indican el estado de validación.
- **(ngSubmit)**: ejecuta una función al enviar el formulario.
- **ReactiveFormsModule**: debe estar importado para utilizar Reactive Forms.

15.2 Ejemplo básico

TS

```
import { Component } from '@angular/core';  
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';  
  
@Component({  
  selector: 'app-formulario',  
  imports: [ReactiveFormsModule],  
  templateUrl: './formulario.html'  
})  
export class FormularioComponent {  
  
  formulario = new FormGroup({  
    nombre: new FormControl('', Validators.required),  
    email: new FormControl('', [Validators.required, Validators.email])  
  });  
  
  enviar() {  
    console.log(this.formulario.value);  
  }  
}
```

HTML

```
<form [formGroup]="formulario" (ngSubmit)="enviar()">  
  <input type="text" formControlName="nombre">  
  <input type="email" formControlName="email">  
  <button type="submit" [disabled]="formulario.invalid">  
    Enviar  
  </button>  
</form>
```



15.3 Elementos para definir y crear un formulario.

1. **FormControl:** representa un campo individual del formulario.

TS

```
nombre = new FormControl('');
```

HTML

```
<input [formControl]="nombre">
```

2. **FormGroup:** Normalmente un formulario tiene varios campos. Para agruparlos utilizamos FormGroup.

TS

```
formulario = new FormGroup({  
  nombre: new FormControl(''),  
  email: new FormControl(''),  
  edad: new FormControl(0)  
});  
// podemos obtener todos los valores  
console.log(this.formulario.value);
```

HTML

```
<form [formGroup]="formulario">  
  
  <label>Nombre:</label>  
  <input formControlName="nombre">  
  
  <label>Email:</label>  
  <input formControlName="email">  
  
  <label>Edad:</label>  
  <input type="number" formControlName="edad">  
  
</form>
```

3. **FormBuilder:** Cuando los formularios empiezan a crecer, escribir muchos new FormControl() puede resultar incómodo. Para ello Angular nos proporciona FormBuilder que es una forma más compacta de crear el mismo formulario

```
import { FormBuilder, Validators } from '@angular/forms';  
  
constructor(private fb: FormBuilder) {}  
  
formulario = this.fb.group({  
  nombre: ['', [  
    Validators.required,  
    Validators.minLength(3)  
  ]],  
  email: ['', [  
    Validators.required,  
    Validators.email  
  ]],  
  edad: [0, [  
    Validators.required,  
    Validators.min(18)  
  ]]  
});
```

O en Angular moderno podemos usar la inyección mediante inject()

```
private fb = inject(FormBuilder);  
  
formulario = this.fb.group({  
  nombre: ['', Validators.required],  
  email: ['', [Validators.required, Validators.email]]  
});
```



15.4 Validación de datos del formulario

Una de las principales ventajas de los formularios reactivos es que las validaciones se pueden definir directamente en TypeScript. Angular proporciona validadores como:

- `Validators.required`
- `Validators.minLength()`
- `Validators.maxLength()`
- `Validators.email`
- `Validators.min()`
- `Validators.max()`
- `Validators.pattern()`

Ejemplo:

```
import { FormControl, FormGroup, Validators } from '@angular/forms';

formulario = new FormGroup({
  nombre: new FormControl('', [
    Validators.required,
    Validators.minLength(3)
  ]),

  email: new FormControl('', [
    Validators.required,
    Validators.email
  ])
});
```

Con estas validaciones le indicamos:

- nombre es obligatorio y debe tener al menos 3 caracteres.
- email es obligatorio y debe tener formato de email

15.5 Mostrar Errores

Podemos comprobar si un campo es válido o inválido con

- `this.formulario.get('email')?.valid`
- `this.formulario.get('email')?.invalid`

15.6 Modificación de valores del formulario desde TypeScript

También podemos modificar los valores del formulario desde TypeScript.

- `setValue()`: Debemos proporcionar todos los campos:
 - `this.formulario.setValue({ nombre: 'Ana', email: 'ana@gmail.com' });`
- `patchValue()`: Permite modificar solamente algunos:
 - `this.formulario.patchValue({ nombre: 'Ana' });`
 -

15.7 Escuchar Cambios en los valores del formulario

Una característica muy interesante es `valueChanges` con la que podemos detectar cada vez que cambia un campo y/o todos los cambios del formulario.

```
this.formulario.get('nombre')?.valueChanges
  .subscribe(valor => {
    console.log('Nuevo nombre:', valor);
  });
```



```
});  
  
this.formulario.valueChanges  
  .subscribe(valor => {  
    console.log(valor);  
  });
```

Esto resulta especialmente útil para:

- Filtrar resultados.
- Habilitar/deshabilitar campos.
- Calcular precios.
- Reaccionar a cambios del usuario.
- Realizar búsquedas.
- Validaciones dinámicas.

15.8 Envío del Formulario

Para el envío del formulario utilizamos (ngSubmit)

Una práctica habitual es validar el formulario antes de realizar el envío

Un ejemplo de uso

HTML

```
<form [formGroup]="formulario" (ngSubmit)="enviar()">  
  
  <input formControlName="nombre">  
  <input formControlName="email">  
  
  <button type="submit">  
    Enviar  
  </button>  
  
</form>
```

TS

```
enviar() {  
  if (this.formulario.valid) {  
    console.log(this.formulario.value);  
  }  
}
```

15.8.1 Posibles estados de un formulario

Propiedad	Significado
valid	El formulario es válido
invalid	El formulario es inválido
pristine	El usuario no lo ha modificado
dirty	El usuario lo ha modificado
touched	El usuario ha entrado/salido del campo
untouched	Todavía no ha sido tocado
value	Valores actuales
errors	Errores de validación

Podemos controlar

```
console.log(this.formulario.value);  
if (this.formulario.invalid) {  
  return;  
}
```



16 Diferencia Formularios plantilla (NgForm) vs Formularios Reactivos

NgForm	Reactive Forms
Basado en la plantilla	Basado en TypeScript
Más sencillo	Más estructurado
ngModel	FormControl
NgForm	FormGroup
Ideal para formularios simples	Ideal para formularios complejos



SECCION 9: Componentes y librerías de terceros



SECCION 10: Extras

17 Alias: Utilidad para mejorar la legibilidad de nuestro código

- Los Alias nos permite configurar alias para referencias direcciones de componentes en el proyecto
- Para su utilización debemos configurarlos el fichero de configuración typeScript.ts
 - Precondición: Tiene que existir la propiedad baseUrl = "/" , si no habrá que incluirla.
 - Después se añade una nueva entrada con la propiedad "path" como la siguiente estructura

```
"Path": {  
  "@alias1/*": ["<ruta desde la raiz>/*"] ,  
  "@alias2/*": ["<ruta desde la raiz>/*"]  
}
```

Ejemplo

- Añades una entrada "path"

```
"path": {  
  "@core/*": ["src/app/core/^*"],  
  "@clientes/*": ["src/modules/clientes/*"],  
  "@personas/*": ["src/modules/personas/*"],  
};
```